# Robbie – "*Object Detected and Moved*"

Aditya Wilson

29 September 2025

## Abstract

This project presents Robbie, a **3-DOF** robotic arm designed to autonomously detect and manipulate objects using low-cost, off-the-shelf components. The system combines computer vision (**YOLOv8 + OpenCV**), embedded motion control (**Arduino Nano**), and visual input from an **ESP32-CAM**. By translating object detection results into inverse kinematics (IK) commands, Robbie demonstrates how a cost-effective robotic manipulator can be constructed for educational and research purposes.

## Introduction

Industrial robotic manipulators are typically expensive and rely on high-precision actuators and sensors, placing them out of reach for many students and hobbyists. Robbie addresses this gap by providing a DIY, low-cost alternative that can be replicated with minimal resources.

Inspired by fictional assistants such as *DUM-E* from *Ironman* (Yes, I am a fan), Robbie was conceived as a hands-on introduction to robotics, machine vision, and kinematic modeling. The initial prototype was constructed from cardboard and broomsticks, later upgraded with 3D-printed parts for precision and durability.

### Key Features:

- Uses widely available microcontrollers (Arduino Nano, ESP32-CAM).
- Custom-trained YOLOv8 object detection model.
- Fully documented workflow for reproducibility.
- Serves as a foundation for more advanced autonomous systems.
- Innovative 3D designed alternatives inspired from tower cranes and LEGO.

# System Architecture

## Input

- ESP32-CAM captures still images of the workspace.
- Python script requests images via HTTP for processing.

## Processing

- YOLOv8 detects objects and outputs bounding box data.
- OpenCV overlays a reference grid and maps object centers.
- Coordinates are transmitted to the Arduino Nano via serial communication.

## Output

- Polar coordinates (r, θ) drive the base stepper rotation and link servos.
- Inverse kinematics compute link angles for target positioning.
- Gripper engages, transfers the object to a drop zone, and releases it.
- Arduino signals completion to the pc as "DONE" to start the next detection cycle.

# Hardware Design

## Actuators

- 28BYJ-48 Stepper Motor – base rotation.
- Two MG996R Servos – control primary arm links.
- SG90 Servo – shovel-style gripper.

## Sensor

- ESP32-CAM (external antenna) – workspace image acquisition.

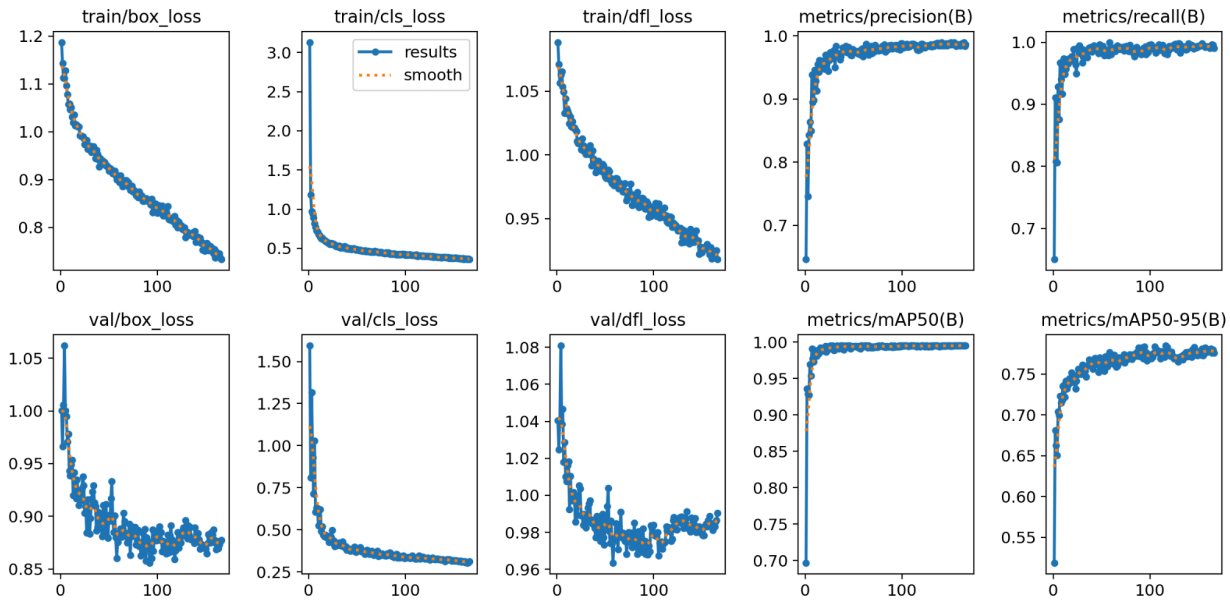
## Materials

- Initial prototype: broomsticks + layered cardboard.
- Final version: 3D-printed components for strength and precision.

# Software Design

## Object Detection Training

- Model: YOLOv8s, custom-trained.
- Dataset: ~100 images per object class, orientations (top, side, bottom).
- Environment: Controlled lighting, white A4 background.
- Training: ~190 epochs, achieving:
    - mAP@50 = 0.995
    - mAP@50-95 ≈ 0.78
    - Precision & Recall ≈ 0.99

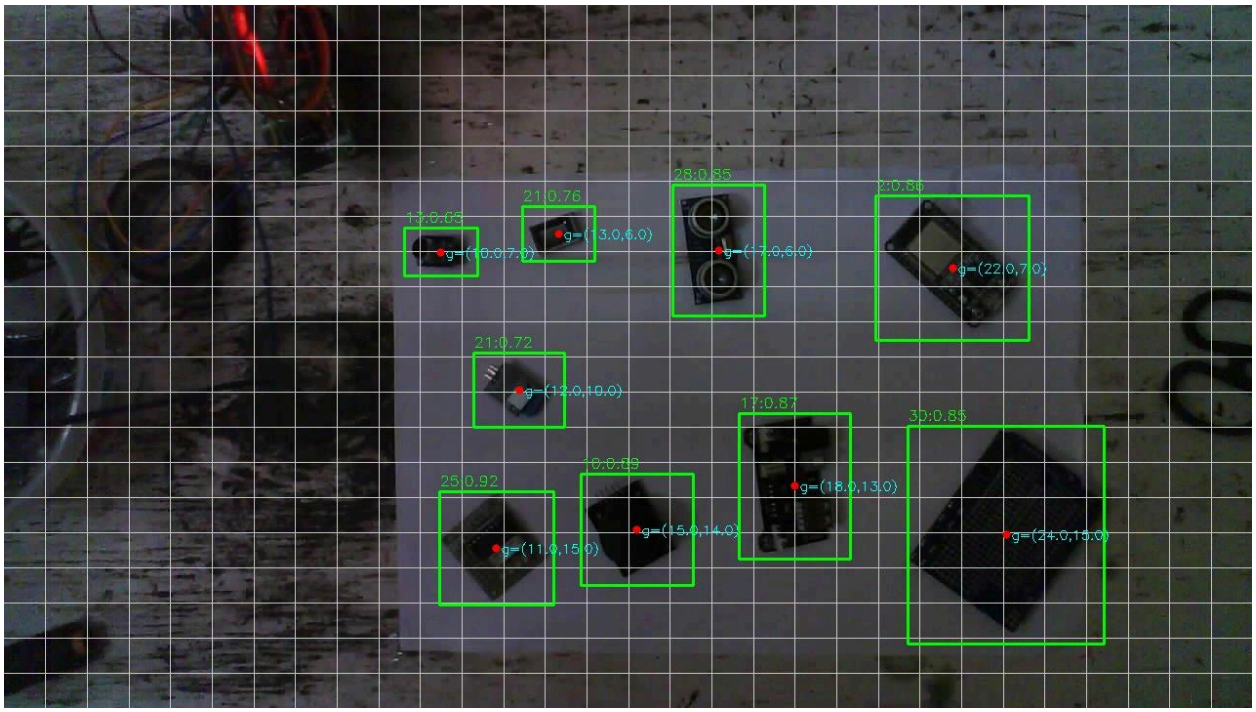



Despite very high accuracy, minor false positives are acceptable; the manipulator can still attempt pickups successfully.

## Python + OpenCV Workflow

- Requests image from ESP32-CAM.
- YOLO processes image → bounding box details saved.
- Object centers mapped to grid coordinates.
- Real-world XY positions sent to Arduino.
- Annotated images optionally saved for visualization.

## Detected Windows Object




## Arduino Actuation

1. Receive Coordinates: XY positions in cm.
2. Translation & Scaling: Apply workspace offsets (transX, transY).
3. Polar Conversion: Convert XY → (r, θ).
4. Inverse Kinematics: Compute joint angles for target reach.
5. Execution:
    - Stepper rotates by θ.
    - Servos move smoothly to target via smoothMove().
    - Gripper closes, transports, and releases the object.
6. Loop: Arduino signals DONE and waits for input from master script.

# Geometry and Kinematics

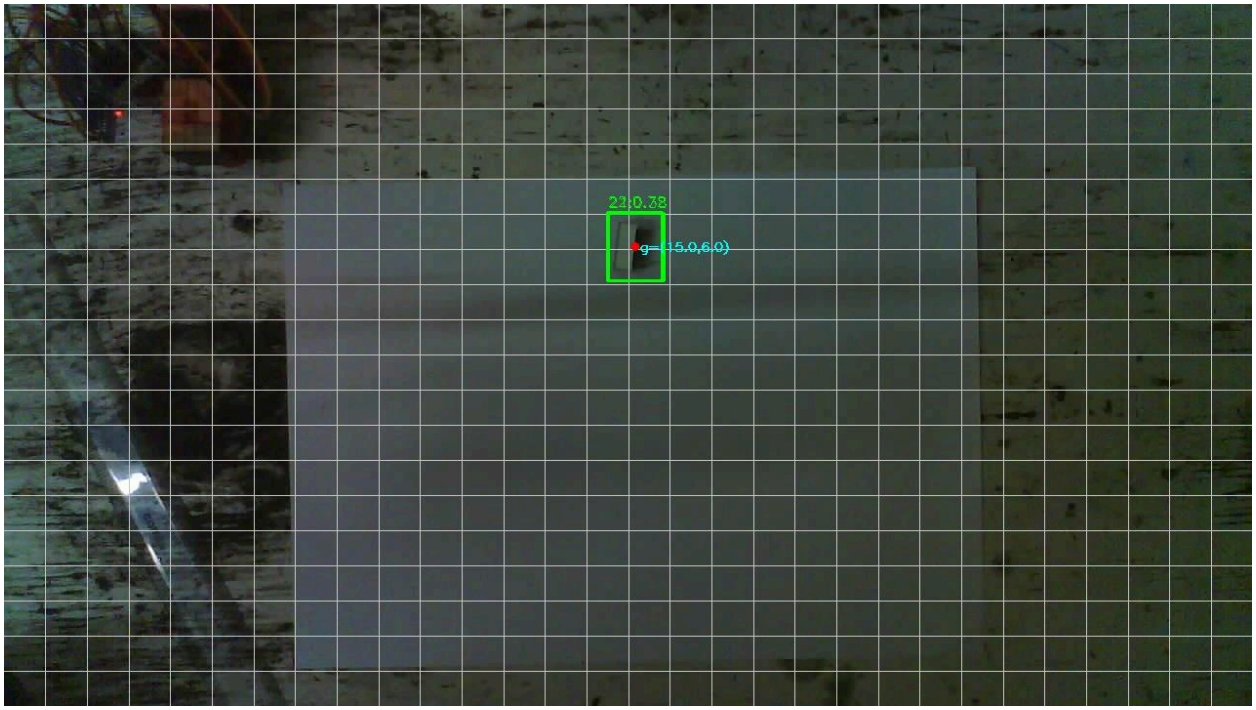
## Key Dimensions

- Base stepper to origin: 3 cm.
- Stepper to workspace edge: ~3 cm.
- Bottom servo height: ~3.5 cm.
- Link 1 (LB): 16 cm.
- Link 2 (LT): 10.5 cm.
- Gripper clearance (GC): 4 cm.

# Kinematic Model

Robbie is modeled as a 2-link planar manipulator with base offset.

- Input coordinates (pixels) → scaled to real-world XY.
- Arduino converts XY → polar (r, θ).
- Inverse kinematics solved using cosine law to compute joint angles.
- Offsets for clearances (SC, GC, LB, LT) included in calculations.




# Derivation for Motion and Guidance

**TO FIND:** Angle CBE and Angle BCD

## Given:

- **AB** is Stepper Height
- **BC** is Bottom Link
- **CD** is Top Link
- **DF** is height of gripper from joint to ground
- **AF** is the distance from base to object

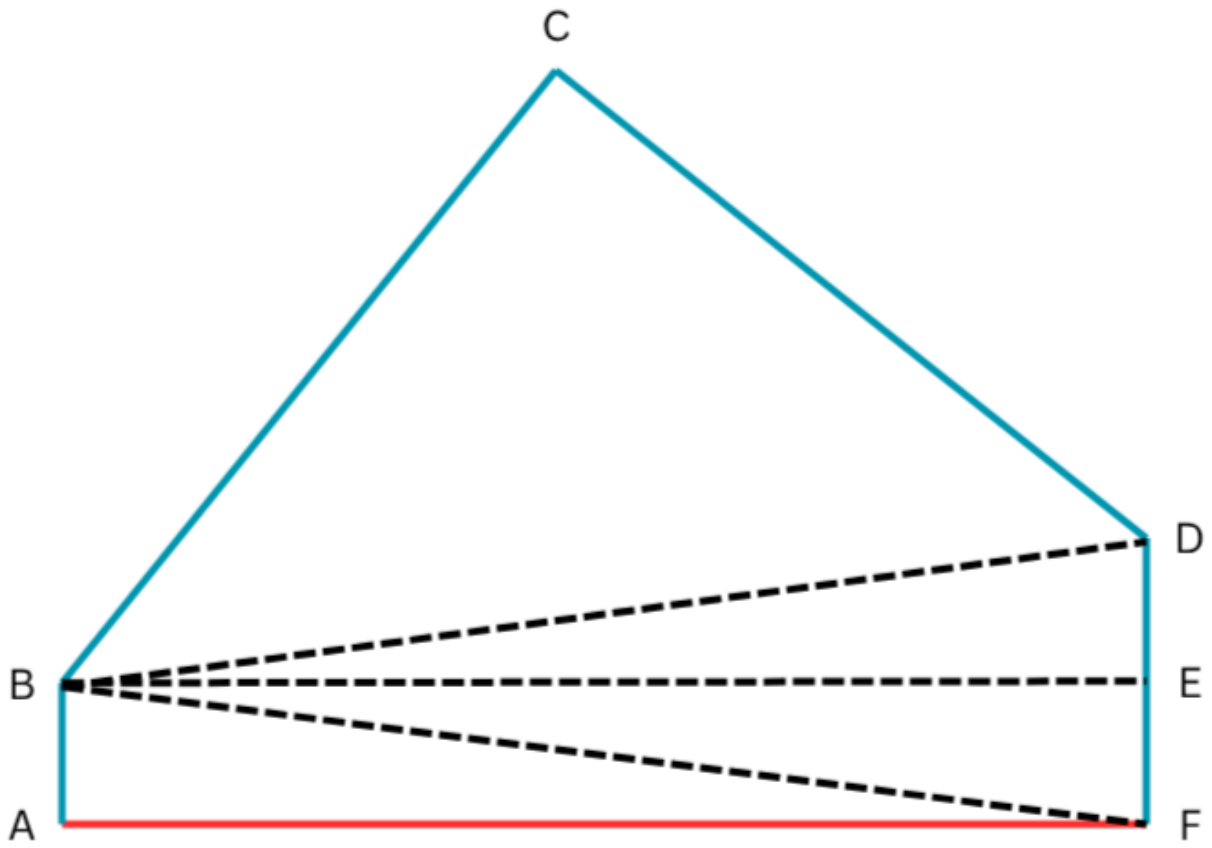
## Extra additions

1. Divide **DF** into **EF** (equals **AB**) and **DE**
2. BE parallel and equal length as **AF**
3. Join **BD** to serve as an “intermediary triangle” during solving

## Order of operations

1. Find **BF** using Pythagoras's Theorem involving **AB** and **AF**
2. Find **BD** using Pythagoras's Theorem involving **BE** and **DE**
3. Find **Angle CBD** using the cosine rule involving **BD**, **BC**, and **CD**
4. Find **Angle DBE** using arctan involving **DE** and **BE**
5. Find **Angle CBE** by subtracting 90 from the sum of **Angles DBE** and **CBD**
6. Find **Angle BCD** using the cosine rule involving **BC**, **BD**, and **CD**

**Note**: In the .ino file, it is given to take the remainder of **Angles DBE** and **CBE**, because those values are for the servo rotation.



# Software Flow Diagram

## Python Script

1. Acquire image.
2. Detect objects (YOLOv8).
3. Map bounding boxes → grid → XY coordinates.
4. Send XY via serial.

## Arduino Code

1. Read XY coordinates.
2. Convert to polar + apply IK.
3. Execute motor and servo motions.
4. Operate gripper.
5. Return to the waiting position.

# Constraints and Safety

- Valid workspace: 7 cm < r < 22.5 cm.
- Base rotation limited to –90° to +180°.
- Motions aborted if inputs exceed limits.
- Servos actuated sequentially to avoid overshoot.
- Stepper operated on an open-loop with step count tracking.

# AI Model Validation

## Ground Truth Annotations (Validation Set):

# Predictions (YOLOv8 Model):


Robbie's trained model performs robustly under real-world lighting variations and clutter. Occasional misclassifications are tolerated since the manipulator operates with redundancy (attempts are still valid picks).

# Mechanical Structure

For modularity, I had used LEGO's plug'n'play method. For structural integrity I decided to use the lattice structure present in tower cranes, inspired by the view on my ride from Vadodara Innovation Council to my house. In the middle there is a hole, which was intended for cables to pass through. Unfortunately at the time of creation of my model I did not have enough cable length and had to have them hang in the air freely, which I admit was a bit unsafe, for which I sealed them with insulation tape.

# Video

# Conclusion

Robbie demonstrates that computer vision-driven robotic manipulation can be achieved with accessible tools and modest hardware. This project merges fundamental EE concepts: control systems, embedded programming, kinematics, and machine learning.

It stands as a practical entry point for students worldwide into robotics research, blending low-cost hardware with cutting-edge AI in an academic framework suitable for undergraduate engineering applications.